

TECHNICAL DESIGN DOCUMENT

RBAC API Service Architecture

Teknologi Stack: NestJS + Prisma 5 + PostgreSQL + JWT + RBAC

Versi Dokumen: 1.0 (Production-Ready)

Tanggal Rilis: 24 Juni 2026

Status: Approved / Terkunci

1. Project Overview

Nama Project: `rbac-service` (Identity & Access Management / IAM)

Tujuan utama dari proyek ini adalah membangun layanan manajemen identitas dan hak akses (*Identity & Access Management Service*) terpusat, modular, dan berperforma tinggi. Service ini bertindak sebagai gerbang otentikasi dan otorisasi terpadu yang akan dikonsumsi oleh berbagai subsistem aplikasi klien melalui antarmuka REST API secara aman dan andal.

Cakupan fungsionalitas inti meliputi:

- **Authentication:** Mekanisme verifikasi identitas menggunakan skema dual-token (Access Token dan Hashed Refresh Token).
- **Authorization:** Pencegatan hak akses pada tingkat routing menggunakan pengondisian berbasis Role dan Permission Guard.
- **Role Management:** Pengelompokan sekumpulan hak akses ke dalam bentuk jabatan/peran pengguna.
- **Permission Management:** Pendefinisian hak akses granular tingkat rendah (*fine-grained*) berbasis resource dan aksi.
- **User Management:** Siklus hidup data pengguna (CRUD) yang terintegrasi dengan penghapusan logis (*Soft Delete*).
- **Audit Logging:** Pencatatan immutable atas seluruh aktivitas mutasi data dan anomali keamanan untuk kebutuhan forensik.

2. Arsitektur Sistem

Aplikasi ini mengadopsi model arsitektur berlapis (*layered architecture*) standar enterprise untuk menjamin isolasi beban kerja, skalabilitas, dan kemudahan pemeliharaan.



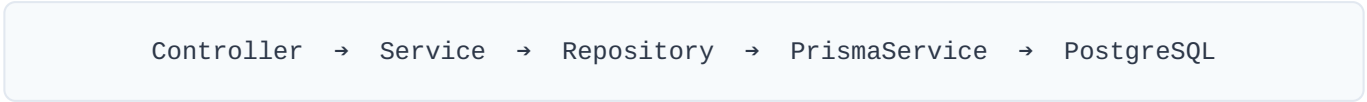
3. Struktur Folder & Prinsip Dependensi

Pola tata kelola berkas wajib mematuhi aturan penempatan direktori **Clean Architecture** berbasis domain modul berikut:

```
src/
├─ config/           # Konfigurasi variable lingkungan (.env) terpusat
├─ common/           # Decorators, Guards, Interceptors, Pipes global
├─ infrastructure/   # Instansiasi PrismaService dan Redis Client
├─ modules/          # Domain Bisnis (auth, users, roles, permissions)
├─ shared/           # Utility, Helpers, dan Constant global
└─ database/         # Skrip Seed data master awal
```

Prinsip Aliran Dependensi (Dependency Rule)

Aplikasi menegakkan aturan aliran dependensi satu arah secara kaku demi kemudahan isolasi **automated testing**:



Ketetapan Absolut: Tidak diperbolehkan melakukan **injection** langsung dari *Controller → PrismaService* atau *Controller → Repository*. Seluruh logika transaksi data wajib melalui abstraksi di dalam layer *Service*.

4. Entity Relationship Diagram (ERD)

Struktur relasi data dalam IAM Service ini memecah hubungan **Many-to-Many** menjadi tabel **explicit junction** guna mendukung performa indeksasi database berskala besar.



5. Spesifikasi Tabel Database

Seluruh entitas menggunakan tipe data biner terindeks UUID untuk mencegah eksploitasi tebakan urutan ID (**enumeration attack**) lintas microservice.

Tabel: users

Nama Field	Tipe Data	Atribut / Keterangan
id	UUID	PRIMARY KEY , default: uuid_generate_v4()
name	VARCHAR(100)	Nullable, Nama lengkap user
email	VARCHAR(150)	UNIQUE , Indeks pencarian utama
password	VARCHAR(255)	Teks terenkripsi Bcrypt Hash
is_active	BOOLEAN	Default: true
last_login	TIMESTAMP	Nullable
created_at	TIMESTAMP	Default: NOW()
updated_at	TIMESTAMP	Auto update on modification
deleted_at	TIMESTAMP	Nullable, Indikator Soft Delete

Tabel: roles

Nama Field	Tipe Data	Atribut / Keterangan
id	UUID	PRIMARY KEY
name	VARCHAR(50)	UNIQUE (Contoh: SUPER_ADMIN)
description	TEXT	Nullable
created_at	TIMESTAMP	Default: NOW()
updated_at	TIMESTAMP	Auto update

Tabel: permissions

Nama Field	Tipe Data	Atribut / Keterangan
id	UUID	PRIMARY KEY
name	VARCHAR(100)	UNIQUE (Format: resource:action)
resource	VARCHAR(50)	Kategori modul (Contoh: user, role)
action	VARCHAR(50)	Jenis aksi (Contoh: create, view)

Tabel: user_roles

Nama Field	Tipe Data	Atribut / Keterangan
id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY -> users(id) ON DELETE CASCADE
role_id	UUID	FOREIGN KEY -> roles(id) ON DELETE CASCADE

Tabel: role_permissions

Nama Field	Tipe Data	Atribut / Keterangan
id	UUID	PRIMARY KEY
role_id	UUID	FOREIGN KEY -> roles(id) ON DELETE CASCADE
permission_id	UUID	FOREIGN KEY -> permissions(id) ON DELETE CASCADE

Tabel: refresh_tokens

Nama Field	Tipe Data	Atribut / Keterangan
id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY -> users(id) ON DELETE CASCADE
token_hash	TEXT	String Hash dari token sesi aktif
expires_at	TIMESTAMP	Batas waktu kedaluwarsa sesi

Tabel: audit_logs

Nama Field	Tipe Data	Atribut / Keterangan
id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY -> users(id) ON DELETE SET NULL
module	VARCHAR(50)	Nama modul yang diakses
action	VARCHAR(100)	Jenis operasi (Contoh: CREATE_USER)
ip_address	VARCHAR(45)	Mendukung format IPv4 dan IPv6

Nama Field	Tipe Data	Atribut / Keterangan
user_agent	TEXT	Metadata aplikasi browser/klien
created_at	TIMESTAMP	Default: NOW() (Immutable)

6. Prisma Naming Convention

Penulisan deklarasi skema Prisma wajib mematuhi standardisasi kapitalisasi guna menjaga konsistensi translasi tipe ke PostgreSQL:

- **Primary Key:** Wajib dideklarasikan sebagai string UUID: `id String @id @default(uuid())`
- **Timestamps:** Menggunakan aturan camelCase pada skema dan dicerminkan sebagai snake_case di database:
`createdAt DateTime @default(now()) @map("created_at")`
`updatedAt DateTime @updatedAt @map("updated_at")`
- **Soft Delete:** Entitas utama wajib memiliki flag tanggal opsional: `deletedAt DateTime? @map("deleted_at")`

7. Permission Standard

Sistem otorisasi IAM menggunakan standarisasi pemetaan hak akses berbasis string datar dengan format formal: *resource:action*.

Resource	Aksi Terdaftar
user	user:create, user:view, user:update, user:delete
role	role:create, role:view, role:update, role:delete
permission	permission:create, permission:view, permission:update, permission:delete
audit	audit:view

8. Default Roles Matrix

1. **SUPER_ADMIN:** Memiliki bypass otomatis atas seluruh skema string permission (*hasAll = true*).
2. **ADMIN:** Diberikan hak penuh untuk manajemen operasional bisnis (CRUD), tidak memiliki izin penghapusan (delete) pada komponen infrastruktur inti.
3. **STAFF:** Hak akses terbatas, didominasi oleh operasi pembacaan data (**:view*) dan pembatasan pembuatan entitas tertentu.

9. API Routing Standard

Seluruh endpoint API wajib diawali dengan prefix versi global untuk mengantisipasi **breaking changes** di masa mendatang tanpa mengganggu klien lama. Base URL ditetapkan sebagai: `/api/v1`.

- Otentikasi: POST `/api/v1/auth/login`, POST `/api/v1/auth/refresh`
- Manajemen User: GET `/api/v1/users`, POST `/api/v1/users`
- Manajemen Role: GET `/api/v1/roles`, PATCH `/api/v1/roles/:id`

10. Response Payload Standard

Success Response

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {}
}
```

Pagination Response

```
{
  "success": true,
  "message": "Data retrieved successfully",
  "data": [],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 100
  }
}
```

Error Response

```
{
  "success": false,
  "message": "Validation Error",
  "errors": [
    "email must be a valid email address",
    "password must be longer than or equal to 8 characters"
  ]
}
```

11. DTO (Data Transfer Object) Standard

Setiap data masuk (*request body* / *query*) wajib diisolasi menggunakan berkas DTO khusus yang menerapkan aturan ketat library `class-validator`. Penamaan berkas diatur sebagai berikut:

- Pembuatan Data: `create-[resource].dto.ts` (Contoh: `create-user.dto.ts`)
- Pembaruan Data: `update-[resource].dto.ts`
- Format Output: `[resource]-response.dto.ts`
- Penyaringan Halaman: `pagination-query.dto.ts`

Ketentuan Validasi Pipe: Seluruh properti di luar DTO akan otomatis dipotong dan request langsung digagalkan (melempar error 400) jika memuat field ilegal (serangan *mass assignment*).

12. JWT (Java Web Token) Standard

- **Access Token:** Umur aktif $T_{\{access\}} = 15 \text{ ext}\{ \text{menit} \}$. Digunakan sebagai otentikasi stateless harian klien.
- **Refresh Token:** Umur aktif $T_{\{refresh\}} = 30 \text{ ext}\{ \text{hari} \}$. Digunakan untuk meminta access token baru.

Payload Struktur Klaim JWT

```
{
  "sub": "b2c34a12-8e12-4c23-9f89-72341aefbcde",
  "email": "admin@example.com",
  "roles": ["SUPER_ADMIN"]
}
```

13. Authentication & Session Flow

Alur Login Utama

User Kirim Kredensial → Validasi Password (Bcrypt) → Generate Access & Refresh Token → Hash Refresh Token via Bcrypt (10 rounds) → Simpan Hash ke Tabel `refresh_tokens` → Return Pasangan Token ke Klien.

Alur Penyegaran Sesi (Token Refresh)

Klien Kirim Plain Refresh Token → Bandingkan String via `bcrypt.compare` dengan Data DB → Jika Valid, Hapus Token Lama (Rotasi Tunggal) → Terbitkan Pasangan Token Baru → Simpan Hash Baru → Return ke Klien.

14. Audit Logging Trigger

Modul log audit bekerja secara independen menggunakan NestJS Interceptor untuk menjamin pencatatan tanpa memblokir proses utama aplikasi. Event mutasi yang wajib direkam meliputi:

- Sesi Keamanan: LOGIN, LOGOUT, AUTH_REFRESH_FAILED
- Pengguna: CREATE_USER, UPDATE_USER, SOFT_DELETE_USER
- Otorisasi: ASSIGN_ROLE_TO_USER, SET_ROLE_PERMISSION

15. Security Hardening Layer

Sistem ini menerapkan prinsip keamanan berlapis (*defense-in-depth*) pada seluruh tingkatan infrastruktur:

Komponen Security	Metode Implementasi	Tujuan Mitigasi
Helmet Middleware	<code>app.use(helmet())</code>	Mencegah serangan Cross-Site Scripting (XSS) & Clickjacking
Rate Limiting	ThrottlerModule + Redis	Mencegah serangan DoS & pemboman brute-force endpoint login
Password Cryptography	<code>bcrypt</code> (12 rounds cost factor)	Perlindungan password di DB terhadap serangan Rainbow Table
Database Isolation	Loopback binding 127.0.0.1:5432	Menutup akses port database langsung dari internet publik
Network Security	UFW Firewall + Fail2Ban	Blokir otomatis IP penyerang port SSH secara real-time

16. Swagger Interface Control

Dokumentasi Swagger API UI terekspos di rute `/api/docs`. Aturan penampilan antarmuka dikunci ketat via variabel lingkungan:

- Aktivasi Penuh: Lingkungan **development** dan **staging** (untuk konsumsi tim frontend dan QA).
- Deaktivasi Absolut: Lingkungan **production**. Blok kode Swagger dilarang diinstansiasi guna mencegah kebocoran visual peta jalan struktur API ke publik.

17. Testing Strategy Matrix

- **Unit Testing:** Fokus pengujian logika murni pada kelas AuthService, UsersService, dan RolesService dengan teknik *mocking* penuh pada layer Repository.

- **End-to-End (E2E) Testing:** Pengujian jalur HTTP nyata pada skenario kritis: rute login, siklus hidup refresh token, dan pembatasan hak akses rute oleh Guard.
- **Target Kepatuhan (Coverage):** Seluruh baris kode kritis wajib menyentuh angka minimal **80% code coverage** sebelum diizinkan masuk ke pipeline rilis.

18. Git Branching Model

Repositori dikelola menggunakan modifikasi pola **Git Flow** standar rilis:

- **main:** Cabang suci, mencerminkan kode stabil yang berjalan di server VPS Production.
- **develop:** Cabang integrasi utama, tempat seluruh fitur baru dikumpulkan dan diuji bersama.
- **feature/*:** Cabang jangka pendek isolasi fitur (Contoh: `feature/auth`, `feature/users`). Wajib melalui **Pull Request (PR)** dan review sebelum menyatu dengan cabang `develop`.

19. Definition of Done (DoD)

Sebuah fitur atau siklus rilis proyek dinyatakan selesai secara mutlak dan layak didorong ke lingkungan Live jika memenuhi kriteria pemenuhan berikut:

1. Seluruh rute CRUD entitas User, Role, dan Permission berfungsi 100% tanpa error.
2. Mekanisme otentikasi dual-token JWT beserta enkripsi Refresh Token lolos uji verifikasi.
3. `PermissionsGuard` sukses memotong dan mencatatkan aksi ilegal ke dalam `AuditLog`.
4. Dokumentasi Swagger terintegrasi dan file Docker multi-stage build berhasil terkompilasi.
5. Pipeline CI/CD GitHub Actions berstatus sukses (**passed**), lulus uji linting, dan otomatis tergelar di VPS Ubuntu melalui reverse proxy Nginx yang terenkripsi SSL aktif.
6. Laporan cakupan pengujian (**automated test coverage report**) menyentuh angka minimal 80%.